

Alterações de esquema

O que a Etapa 2 exigiu acrescentar ao banco da Etapa 1

Por que o esquema teve de mudar

Três requisitos da Etapa 2 pedem informação que o banco da Etapa 1 não guarda. A view `vw_pacientes_internados` descreve pacientes internados a partir de uma data de saída nula, e nenhuma tabela do esquema original registra internação. A view `vw_estatisticas_atendimentos_mensal` agrega por mês e por unidade, mas `ATENDIMENTO` não tem ligação com `UNIDADE`: no modelo da Etapa 1, unidade só aparece em `ESCALA`. E `sp_calcular_tempo_medio_espera` compara a chegada do paciente com o início do primeiro procedimento, informação que `PROCEDIMENTO_REALIZADO` não armazenava, já que ali só há duração, quantidade e observação.

As alterações ficaram em `05_etapa2_estrutura.sql`, um arquivo separado. O `01_schema.sql` não foi tocado, então a entrega da Etapa 1 continua reproduzível exatamente como foi avaliada, e o histórico do repositório mostra onde uma etapa termina e a outra começa.

Resumo das alterações

Objeto	Alteração	Motivo
atendimento	coluna <code>id_unidade</code> , FK para unidade, opcional	as views e o cálculo de espera agrupam por unidade
procedimento_realizado	coluna <code>data_hora_inicio</code>	medir a espera até o primeiro procedimento
procedimento	coluna <code>media_tempo_procedimento</code>	valor derivado mantido por trigger
escala	coluna <code>versao</code> , NOT NULL, padrão 1	controle de concorrência otimista
internacao	tabela nova	base da <code>vw_pacientes_internados</code>
auditoria_atendimento	tabela nova	destino do <code>trg_audita_atendimento</code>

Decisões que merecem explicação

`id_unidade` em `atendimento` aceita nulo

A coluna nasceu opcional. Os endpoints em SQL puro da Etapa 1 inserem atendimento sem informar unidade, e declarar a coluna como NOT NULL sem valor padrão quebraria essas rotas. Como a Etapa 1 continua no ar para comparação, a coluna aceita nulo e as views ignoram as linhas incompletas. A camada ORM da Etapa 2 pede o campo no formulário.

O efeito prático aparece na interface: com a chave do cabeçalho em Etapa 1, um atendimento criado pela tela fica sem unidade e não entra nas estatísticas mensais. Com a chave em Etapa 2, entra.

Um paciente não pode ter duas internações abertas

A regra está no banco, num índice único parcial:

05_etapa2_estrutura.sql

```
CREATE UNIQUE INDEX uq_internacao_aberta
ON Internacao(id_paciente)
WHERE data_hora_saida IS NULL;
```

A cláusula WHERE limita a restrição às internações em aberto. Sem ela, um índice único em id_paciente impediria o paciente de ser internado uma segunda vez na vida, o que não é a regra desejada. Com ela, o histórico de altas fica livre e apenas a simultaneidade é barrada. Isso também deixa a vw_pacientes_internados sem ambiguidade: no máximo uma internação aberta por paciente.

A auditoria não tem chave estrangeira para atendimento

auditoria_atendimento.id_atendimento é um INTEGER comum, sem REFERENCES. O trigger registra também as remoções. Com ON DELETE CASCADE, apagar um atendimento levaria embora a linha de auditoria que registra essa remoção. Sem cascata, a chave estrangeira impediria o DELETE. Nenhuma das duas alternativas serve para uma tabela de auditoria, então a coluna ficou sem referência declarada.

A média de tempo é uma coluna, não uma consulta

media_tempo_procedimento guarda um valor que poderia ser calculado a qualquer momento com AVG. Guardar troca custo de leitura por custo de escrita: as listagens de procedimento não precisam agregar nada, e em contrapartida cada procedimento realizado provoca um recálculo. O enunciado pede a coluna e o trigger que a mantém, então a escolha vem de lá; o efeito colateral está descrito no documento sobre triggers.

Os procedimentos já gravados pelo seed da Etapa 1 ficariam com a coluna vazia, porque o trigger só age a partir da sua criação. O 05_etapa2_estrutura.sql termina com um UPDATE que calcula o valor inicial a partir dos dados existentes.

Ordem de execução

Os scripts do PostgreSQL em /docker-entrypoint-initdb.d rodam em ordem alfabética, e é isso que define a numeração dos arquivos. O docker-compose.yml monta os seis que alteram o banco:

Arquivo	Papel	Roda no docker compose up
01_schema.sql	tabelas da Etapa 1	sim
02_seed.sql	dados de teste da Etapa 1	sim
04_analiticas.sql	as 4 consultas analíticas	não, só SELECT
05_etapa2_estrutura.sql	tabelas e colunas novas	sim
06_etapa2_procedures.sql	3 stored procedures	sim
07_etapa2_triggers.sql	3 triggers	sim
08_etapa2_views.sql	3 views	sim
09_etapa2_seed.sql	dados de teste da Etapa 2	sim
10_etapa2_verificacao.sql	roteiro de verificação	não, só SELECT

Os triggers são criados antes do seed da Etapa 2. Assim as inserções do 09 passam por eles: a auditoria nasce com quinze linhas e as médias de procedimento saem calculadas pelo caminho normal, sem precisar de um UPDATE à parte.

Os scripts usam IF NOT EXISTS em colunas, tabelas e índices, e DROP antes de CREATE nas rotinas e views. Rodá-los duas vezes no mesmo banco não dá erro, o que é útil enquanto se ajusta alguma coisa sem recriar o volume do contêiner.

O que não mudou

- Nenhuma tabela ou coluna da Etapa 1 foi removida ou renomeada.
- Nenhuma constraint existente foi afrouxada. A UNIQUE de escala continua igual, e o trigger da Etapa 2 apenas soma uma regra a ela.
- Os endpoints da Etapa 1 continuam respondendo o mesmo JSON. Os campos novos ficaram em schemas Pydantic separados, usados só pelas rotas em /orm.
- O seed da Etapa 1 não foi editado. O 09_etapa2_seed.sql preenche as colunas novas das linhas antigas casando por data e hora, não por id.